TEST PROJECT
LOMBA KOMPETENSI SISWA SMK TINGKAT KOTA BEKASI TAHUN 2026

MODUL A - LINUX ENVIRONMENT

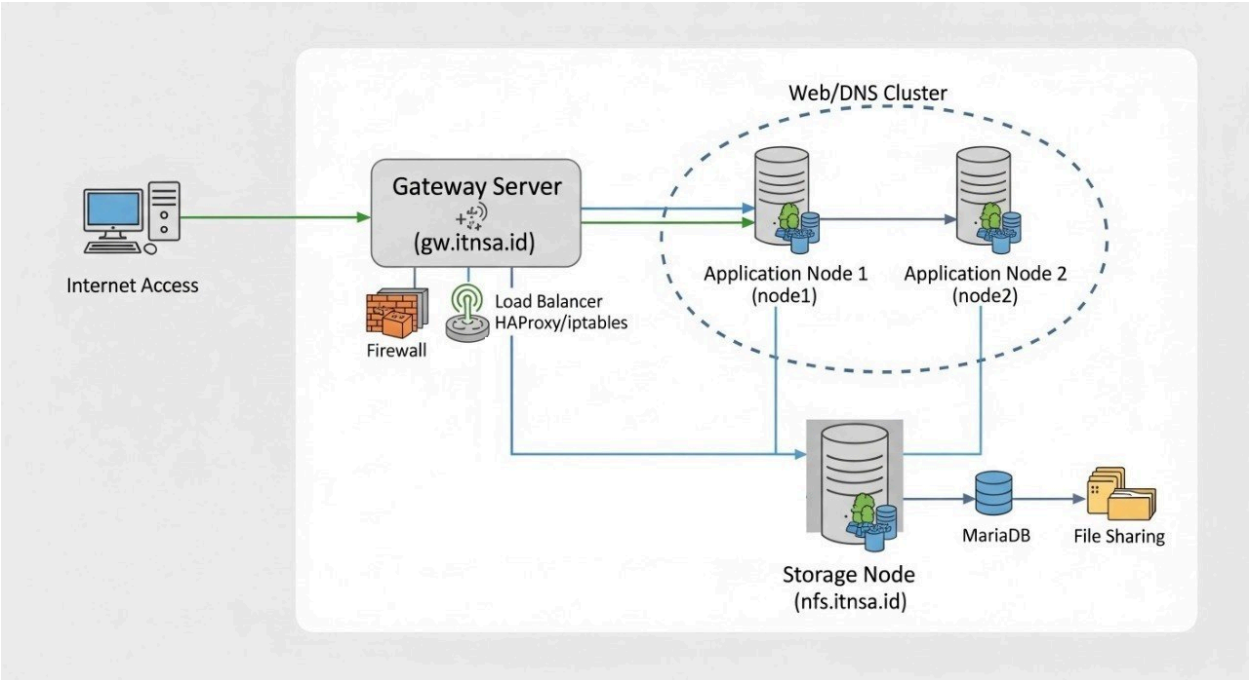
MODUL IT NETWORK SYSTEMS ADMINISTRATION

Introduction

This document outlines Modul A: Linux Environment, a core component of the Lomba Kompetensi Siswa (LKS) SMK Tingkat Kota Bekasi Tahun 2026, specifically for the IT Network Systems Administration skill area.

Project Scope and Objective:
The primary objective of this project is to challenge participants to design, implement, and manage a highly available (HA) and secure network infrastructure. The solution must be built upon a 4-server topology utilizing Debian 13 (Trixie) within a simulated cloud environment. A critical constraint is the exclusive use of Public IP addresses for all internal and external communication, necessitating stringent firewall and security policies.

Topology and Server Roles



The network is segmented into distinct functional roles:

- **Gateway Server (gw.itnsa.id):** Serves as the network edge, managing Security (iptables Firewall), Traffic Distribution (HAProxy Load Balancer), Public-facing Communication (Mail Server), and acting as the Ansible Control Node for automation.
- **Application Nodes (node1.itnsa.id & node2.itnsa.id):** These two servers form a cluster to provide High Availability for core services, including clustered Web Services (Nginx), Primary/Slave DNS (BIND9), and scheduled data backup.
- **Storage Node (nfs.itnsa.id):** Functions as the central repository for shared data, hosting the MariaDB Database Server and providing essential File Sharing services (NFS, Samba, FTP).

Challenge Requirements:
Participants are required to follow detailed manual configuration steps (Part 1, 2, 3, 4) to ensure all installations and security measures are uniformly applied. An optional, high-value challenge involves creating idempotent Ansible playbooks to automate the entire setup, rewarding efficiency and mastery of infrastructure as code.

Login & Operating System

- **OS:** Debian 13 (Trixie)
- **Username:** root
- **Password:** LksBekasi2026!

System Configuration

Timezone: Asia/Jakarta

General Configuration Table:

This table provides a comprehensive overview of the Fully Qualified Domain Name, Network configuration, and core services running on each of the four designated servers.

Fully Qualified Domain Name	IPv4 / Network	Services
gw.itnsa.id	WAN: (Public IP 1)	Firewall (iptables), Load Balancer (HAProxy), CA (openssl), Mail Server (postfix + dovecot), Ansible Control Node
node1.itnsa.id	WAN: (Public IP 2)	Web Server 1 (nginx), DNS Primary (bind9), Scheduled Backup (rsync)

Fully Qualified Domain Name	IPv4 / Network	Services
node2.itnsa.id	WAN: (Public IP 3)	Web Server 2 (nginx), DNS Slave (bind9), Scheduled Backup (rsync)
nfs.itnsa.id	WAN: (Public IP 4)	File Sharing (NFS, Samba, FTP), MariaDB Database Server

Required User Accounts

All new users must use the password: **LksBekasi2026!**

User	Role	Used On Server	Notes
admin_lks	System Administrator	gw, node1, node2, nfs	Must have sudo privileges for maintenance tasks.
email_test1	Mail Test User 1	gw.itnsa.id	Used for email sending/receiving tests.
email_test2	Mail Test User 2	gw.itnsa.id	Used for email sending/receiving tests.
file_share	File Access User	nfs.itnsa.id	Accesses /srv/data_public for file sharing.
db_admin	MariaDB Database User	nfs.itnsa.id	User for database administration and application access.

(Note: The testing client is each participant's PC/Laptop connected via the Internet).

Part 1: Core Infrastructure

This section details the essential, manual configuration steps required for each of the four servers (Gateway, Storage Node, and two Application Nodes) to establish the basic, secure, and networked environment.

1. Configuration Tasks

1. gw.itnsa.id (Gateway & Control)

- **General Configuration:** Configure hostname in /etc/hostname, static host entries in /etc/hosts, and NTP to use id.pool.ntp.org while enforcing Asia/Jakarta timezone.
- **Certificate Authority (CA):** Create a Self-Signed Root CA ("Bekasi LKS 2026 CA") and sign certificates for *.itnsa.id and mail.itnsa.id. All CA files and signed certificates must be stored in /etc/ssl/itnsa_ca/.
- **Email Server:** Install Postfix (listening on all interfaces) and Dovecot; enable IMAPS (port 993) and secure SMTP submission (port 587) with TLS using the CA certificates. Create local users: **email_test1** and **email_test2**.
- **Firewall:** Configure iptables to allow only necessary public ports (80, 443, 25, 587, 993, 53) and explicitly block external access to backend services like MariaDB (3306) and NFS, allowing traffic only from Gateway/Application Nodes.
- **Load Balancer (HAProxy):** Configure Round Robin load balancing for HTTP/HTTPS traffic to node1 and node2, including TLS termination with the CA certificate.

2. nfs.itnsa.id (Storage Node)

- **General Configuration:** Configure hostname in /etc/hostname, static host entries in /etc/hosts, and NTP to use id.pool.ntp.org while enforcing Asia/Jakarta timezone.

3. node1.itnsa.id & node2.itnsa.id (Application Nodes)

- **General Configuration:** Configure hostname in /etc/hostname, static host entries in /etc/hosts, and NTP to use id.pool.ntp.org while enforcing Asia/Jakarta timezone.

2. Optional Automation Challenge (Ansible)

Ansible Automation is an **Optional Challenge**. Participants may choose to configure each server manually. Successfully automating the configuration using Ansible (playbooks stored in /etc/ansible, must be idempotent) will earn a higher score.

- **01-general.yml:** Automate basic configuration (NTP using id.pool.ntp.org, hostname, /etc/hosts) to node1, node2, and nfs. Ensure the timezone module is used to enforce Asia/Jakarta.
- **02-users.yml:** Automate bulk creation, modification, and deletion of users using variables.
- **03-dns.yml:** Automate installation and troubleshooting (auto-restart if service fails) for BIND9 on node1 and node2.
- **04-web.yml:** Automate the installation of Nginx on node1 and node2, and MariaDB on nfs.

Part 2: Database, Storage & File Services

nfs.itnsa.id (Storage Node) This server provides centralized file sharing. You must secure access using firewall rules (iptables) to only allow traffic from specific public IPs.

1. **MariaDB Database Server**
    - **Installation:** Install the MariaDB Server package.

- **Configuration:** Configure MariaDB to listen on its public IP address (WAN: Public IP 4). This requires editing the bind-address directive in the configuration file (e.g., /etc/mysql/mariadb.conf.d/50-server.cnf).
- **Initial Setup:** Secure the MariaDB installation (mysql_secure_installation) and create a required application database (e.g., app_db).
- **User Permissions:** Create the **db_admin** user (as defined in **Required User Accounts**) and grant all necessary permissions for the app_db database, allowing connections only from the public IPs of **gw.itnsa.id**, **node1.itnsa.id**, and **node2.itnsa.id**.
- **Firewall Security:** Configure **iptables** to allow incoming traffic on MariaDB port (TCP 3306) ONLY from the public IPs of **gw.itnsa.id**, **node1.itnsa.id**, and **node2.itnsa.id**.

2. **File Sharing Services**
    - **NFS:** Export /srv/web_data as an NFS share. Configure /etc/exports to restrict access to the public IPs of **node1.itnsa.id** and **node2.itnsa.id** with rw,sync,no_root_squash options.
    - **Samba & FTP:** Create a Samba share named [public_data] pointing to /srv/data_public. For FTP, use **vsftpd** and ensure the **file_share** user is jailed (chrooted) to their home directory.

Part 3: Application Nodes & Clustering

node1.itnsa.id & node2.itnsa.id These nodes run the application and DNS services. All inter-node communication must happen via Public IPs securely.

1. DNS Server (Primary & Slave)

- **node1.itnsa.id:** Configure BIND9 as master. Place the zone file at /var/lib/bind/db.itnsa.id. Ensure the allow-transfer directive is limited to node2's public IP.
- **node2.itnsa.id:** Configure BIND9 as Slave DNS receiving updates from node1.itnsa.id.

2. Web Services & Database Cluster

- **NFS Mount:** Configure /etc/fstab to mount nfs.itnsa.id:/srv/web_data to /var/www/website using options hard,intr.
- **Nginx:** Create a virtual host at /etc/nginx/sites-available/www.itnsa.id listening on ports 80 and 443. Point the root to the NFS mount at /var/www/website.
- **MariaDB:** Access the MariaDB Database Server installed on nfs.itnsa.id from node1 and node2 using the **db_admin** user.
- **WordPress Installation:** Download the latest WordPress package, extract it into the NFS-mounted directory (/var/www/website), configure the wp-config.php file to connect to the app_db database on nfs.itnsa.id using the **db_admin** user, and complete the installation via the web interface.

3. Scheduled Backup

- Create a Cron job in /etc/cron.d/backup-rsync that runs daily at 02:00 AM. Use rsync -avz to synchronize /var/www/website to a backup directory on the peer node.
- **MariaDB Backup:** Create a Cron job on node1 and node2 to perform a daily backup of the app_db database from nfs.itnsa.id to a local directory using the **db_admin** user.

Part 4: Edge Security & Load Balancing

gw.itnsa.id (Gateway)

1. Firewall (iptables)

- Configure iptables with a default drop policy. Explicitly allow ports 80, 443, 25, 587, 993, and 53 (TCP/UDP) in the INPUT chain.
- Secure internal services by blocking direct internet access to backend application ports (e.g., MariaDB 3306 and NFS); allow routing only from the Gateway IP or Application Nodes.

2. Load Balancer

- In /etc/haproxy/haproxy.cfg, define frontends for HTTP/HTTPS and a backend using balance roundrobin with health checks.
- Apply TLS termination in HAProxy using the Self-Signed CA certificate.

Part 5: Advanced Automation Challenge

This optional section is designed to test advanced scripting and Infrastructure as Code (IaC) skills. Successful implementation yields maximum bonus points. All automation must be initiated from the Ansible Control Node (gw.itnsa.id).

1. Bulk User Provisioning and PAM Authentication

This task requires the creation and execution of an idempotent script (Ansible Playbook or Bash Script) that performs the following:

- **User Creation:** Create exactly 100 new local users named user001 through user100 on both node1.itnsa.id and node2.itnsa.id.
- **User Parameters:** All users must have their primary group set to lks_challenge (must be created if non-existent). The password for all users must be identical: **LksBekasi2026!**.
- **PAM Configuration:** Implement a strict PAM configuration on both node1.itnsa.id and node2.itnsa.id to enforce a strong password policy for all subsequent password changes (e.g.,

utilizing pam_cracklib or pam_pwquality to require a minimum length of 12 characters, a mix of uppercase, lowercase, numbers, and symbols).

- **Verification:** The script must include a check to verify that all 100 users were created and the PAM configuration is active.

2. Automated Service Management and Health Check

Create an Ansible Playbook (05-services.yml) or a comprehensive service management script that allows for:

- **Status Check:** A single command execution that provides the operational status (active, inactive, failed) for all core services across ALL four servers:
  - **gw.itnsa.id:** Postfix, Dovecot, HAProxy, iptables (Check if the service is running or if the configuration is loaded).
  - **node1.itnsa.id & node2.itnsa.id:** Nginx, BIND9, Cron.
  - **nfs.itnsa.id:** MariaDB, NFS, Samba, vsftpd.
- **Selective Restart:** The ability to selectively restart a specified service on a specified target node (or group of nodes) from the Ansible Control Node via a single command (e.g., restart Nginx on node1). The command must only restart the target service and not affect any other running service.